

SamakMarket - Guide Complet

Marketplace locale de vente de poisson — sans e-commerce.

Les acheteurs contactent les vendeurs directement via téléphone ou WhatsApp.

STRUCTURE DU PROJET

SamakMarket/

- |— database.sql # Script SQL création base de données
- |— backend/ # API Node.js + Express
 - |— .env.example # Variables d'environnement (copier en .env)
 - |— package.json
 - |— server.js # ⚡ Point d'entrée principal
 - |— config/
 - |— database.js # Connexion MySQL via Sequelize
 - |— models/ # Modèles Sequelize (représentent les tables)
 - |— index.js # Import + associations entre modèles
 - |— User.js # Utilisateurs (admin/seller/buyer)
 - |— Product.js # Produits des vendeurs
 - |— Ad.js # Publicités payantes
 - |— Subscription.js # Abonnements SaaS
 - |— Review.js # Avis produits
 - |— controllers/ # Logique métier (ce que fait chaque route)
 - |— authController.js # Inscription, connexion, profil
 - |— productController.js # CRUD produits
 - |— adController.js # Gestion publicités
 - |— routes/ # Définition des URLs de l'API
 - |— auth.js # /api/auth/*
 - |— products.js # /api/products/*
 - |— ads.js # /api/ads/*
 - |— middleware/
 - |— auth.js # Vérification JWT + rôles
- |— frontend/ # Application Vue.js
 - |— src/
 - |— services/
 - |— api.js # Instance Axios configurée

```
|— stores/
|   |— auth.js      # Pinia store (état connexion)
|— views/
|   |— MarketplaceView.vue # Page principale
|— components/
|   |— ProductCard.vue   # Carte produit
```

INSTALLATION

Backend

```
bash

cd backend
cp .env.example .env      # Créer le fichier .env
nano .env                 # Remplir vos valeurs DB et JWT_SECRET

npm install               # Installer les dépendances
```

Base de données

```
bash

# Créer la base de données
mysql -u root -p < ../database.sql
```

Lancer le serveur

```
bash

npm run dev              # Mode développement (nodemon)
npm start                 # Mode production
```

Frontend

```
bash

cd frontend
npm create vite@latest . -- --template vue
npm install axios pinia
npm install -D tailwindcss
npm run dev
```

Authentification

Méthode	URL	Auth	Description
POST	<code>/api/auth/register</code>	✗	Inscription
POST	<code>/api/auth/login</code>	✗	Connexion → reçoit le token
GET	<code>/api/auth/me</code>	✓	Mon profil

Produits

Méthode	URL	Auth	Description
GET	<code>/api/products</code>	✗	Tous les produits (avec filtres)
GET	<code>/api/products/:id</code>	✗	Un produit
POST	<code>/api/products</code>	Vendeur	Créer un produit
PUT	<code>/api/products/:id</code>	Vendeur	Modifier
DELETE	<code>/api/products/:id</code>	Vendeur	Supprimer
GET	<code>/api/products/seller/mine</code>	Vendeur	Mes produits

Publicités

Méthode	URL	Auth	Description
GET	<code>/api/ads?position=homepage</code>	✗	Pubs actives par position
POST	<code>/api/ads</code>	Vendeur	Soumettre une pub
PUT	<code>/api/ads/:id/status</code>	Admin	Valider/rejeter
POST	<code>/api/ads/:id/click</code>	✗	Tracker un clic

💡 CONCEPTS EXPLIQUÉS

MVC (Model-View-Controller)

- **Model** = structure de la donnée (Sequelize → MySQL)
- **View** = ce que voit l'utilisateur (Vue.js frontend)
- **Controller** = la logique entre les deux (Node.js)

JWT (JSON Web Token)

Token que l'API renvoie après login.

Le frontend le stocke et l'envoie dans chaque requête :

Authorization: Bearer eyJhbGc...

bcrypt

Algorithme de hachage pour les mots de passe. `password123` devient `$2b$10$Xk9mN...` (irréversible).

Sequelize Associations

User hasMany Products → Un vendeur a plusieurs produits
Product belongsTo User → Un produit appartient à un vendeur

💰 BUSINESS MODEL

Revenus actuels

1. **Produits sponsorisés** (is_featured) → apparaissent en premier dans la liste
2. **Publicités bannière** (table ads) → homepage, sidebar, top
3. **Abonnements premium** (table subscriptions) → plus de produits, badge vendeur pro

Plans suggérés

Plan	Prix/mois	Produits max	Pub	Badge
Free	0	5	✗	✗
Basic	5€	20	✗	✗
Premium	15€	100	✓	✓
Pro	30€	∞	✓	★

CONSEILS SCALABILITÉ

Court terme

- Ajouter les **migrations Sequelize** (plutôt que sync)
- Utiliser **Redis** pour le cache des listes produits
- Ajouter un **rate limiting** (express-rate-limit)

Moyen terme

- Intégrer **Cloudinary** pour le stockage d'images
- Ajouter **Socket.io** pour un chat en temps réel
- Créer un **panel admin** complet

Long terme

- Passer à une architecture **microservices**
- Ajouter un **système de paiement** (Stripe / Cinetpay pour Afrique)
- Créer une **app mobile** (React Native ou Flutter)

SÉCURITÉ

- ☒ Mots de passe hashés (bcrypt)
- ☒ Authentification JWT
- ☒ Vérification des rôles (admin/seller/buyer)
- ☒ Vérification propriétaire avant modification
- ☒ Variables sensibles en .env (jamais en dur dans le code)

À ajouter :

- Rate limiting sur les routes d'auth
- Validation stricte des entrées (express-validator)
- Helmet.js (headers HTTP sécurisés)
- CORS restrictif en production